Concepts of Programming Languages

Functions & Functional Abstraction

Prof. Dr. Mira Mezini

Software Technology Group
TU Darmstadt | FB Informatik

A Note on Abstraction

- Abstraction is a powerful concept in Computer Science
  - “Hide details”
  - “Make smth generic w.r.t. smth else”
  - “Parametrize over smth”

- let-expressions: Parametrize an expression over another expression

- Generics: Parametrize over types

- Functions: Parametrize over … ?

A Taxonomy of Functions

- **First-order functions**
  - Parameterise computations over data
  - Functions that neither return nor take other functions as parameters
- **Higher-order functions**
  - Functions that return and/or take other functions as parameters
  - Parameterize computations over other computations
- **First-class functions**
  - Functions are values with all the rights of other values
  - Can be constructed at runtime
  - Can be passed as arguments to and returned by other functions, stored in data structures etc.

A Taxonomy of Functions

No first-class functions =>

functions can only be defined in designated portions of the program, where they are given names for use in the rest of the program.

A Taxonomy of Functions

- **First-order Functions**
  - Functions that neither return nor take other functions as parameters
  - Parameterise computations over data
- **Higher-order functions**
  - Functions that return and/or take other functions as parameters
  - Parameterize computations over other computations
- **First-class functions**
  - Functions are values with all the rights of other values

Question:
Do we need the distinction between higher-order and first-class functions? Does higher-order imply first-class?

Higher-Order versus First-Class

Some languages achieve some kind of higher-orderness without first-class functions

- Functions pointers (C, C++)
- Eval / Execute
- Objects with a “call” method
  - “A function is an object with a single method”
  - Methods of objects (“functions”) can be parameterized by other objects, which have methods that can be called

Languages With First-Class Functions

- Languages typically associated with functional programming, such as Lisp, Scheme, ML, and Haskell, all support first-class functions.

- Other languages which also support them include Scala , Perl, Python, PHP, Lua, Tcl/Tk, ECMAScript (JavaScript, ActionScript), Ruby.

Outline

- **Implementing first-order functions**
- Environments (static vs. dynamic scoping)
- About first-class functions
- Functional decomposition and recursion patterns

F1LAE

- A language with **first-order** functions
  - Function **applications are expressions**
  - Function **definitions are not expressions**

- No first-class/higher-order functions

⇒ function definitions are separated from expressions

⇒ function definitions given to the interpreter as an argument

Concrete & Abstract Syntax for F1LAE

```
<F1LAE> ::= <num>
          | {+ <F1LAE> <F1LAE>}
          | {- <F1LAE> <F1LAE>}
          | {let {<id> <F1LAE>} <F1LAE>}
          | <id>
          | ???
```

how does the concrete syntax change?

```
sealed trait F1LAE
case class Num(n: Int) extends F1LAE
case class Add(lhs: F1LAE, rhs: F1LAE) extends F1LAE
case class Sub(lhs: F1LAE, rhs: F1LAE) extends F1LAE
case class Let(name: String, namedExpr: F1LAE, body: F1LAE) extends F1LAE
case class Id(name: String) extends F1LAE

???
```

how does the abstract syntax change?

Concrete & Abstract Syntax for F1LAE

```
<F1LAE> ::= <num>
          | {+ <F1LAE> <F1LAE>}
          | {- <F1LAE> <F1LAE>}
          | {let {<id> <F1LAE>} <F1LAE>}
          | <id>
          | {<id> <F1LAE>}
```

concrete syntax for function application

What does this tell us about valid F1LAE programs?

```
sealed trait F1LAE
case class Num(n: Int) extends F1LAE
case class Add(lhs: F1LAE, rhs: F1LAE) extends F1LAE
case class Sub(lhs: F1LAE, rhs: F1LAE) extends F1LAE
case class Let(name: String, namedExpr: F1LAE, body: F1LAE) extends F1LAE
case class Id(name: String) extends F1LAE

case class App(funName: String, arg: F1LAE) extends F1LAE
```

abstract syntax for function application

F1LAE

- We cannot define new functions in F1LAE.
  Function definitions are not expressions.
  There is no expression of the kind (**`fun`** ...)

- Function applications can be nested.

- Function applications can be bound to names in let expressions.

- Function Ids can be bound to names in let expressions.

Function Definitions in F1LAE

- We represent function definitions as a structure **`FunDef`**

```
case class FunDef(argName: String, body: F1LAE)
```

- Example

```
FunDef("n", Add("n", Num(1)))
```

- Predefined functions are passed to the interpreter in a map of ids to function definitions.

```
type FunDefs = Map[String, FunDef]
```

Example Interpreter Calls

```
interp(
   App("f", 10),
   Map("f" -> FunDef("n", App("n", "n")))
)
```

```
interp(
   App("f", 10),
   Map(
     "f" -> FunDef("x", App("g", Add("x", 3))),
     "g" -> FunDef("y", Sub("y", 1)))
```

Discussion

<Interpreter>

What is needed to interpret F1LAE expressions?

How would you interpret App expressions?

F1LAE Interpreter

```
def interp(expr: F1LAE, funDefs: Map[String, FunDef] ): Int = expr match {
  case Num(n) = n
  case Add(lhs, rhs) => interp(lhs, funDefs) + interp(rhs, funDefs)
  case Sub(lhs, rhs) => interp(lhs, funDefs) - interp(rhs, funDefs)

  case Let(id, expr, body) =>
    val body = subst(body, id, Num(interp(expr, funDefs)))
    interp(body, funDefs)

  case Id(name) => sys.error("found unbound id " + name)

  case App(fun, arg) =>  ???



}
```

F1LAE Interpreter

```
def interp(expr: F1LAE, funDefs: Map[String, FunDef] ): Int = expr match {
  case Num(n) = n
  case Add(lhs, rhs) => interp(lhs, funDefs) + interp(rhs, funDefs)
  case Sub(lhs, rhs) => interp(lhs, funDefs) - interp(rhs, funDefs)

  case Let(id, expr, body) =>
    val body = subst(body, id, Num(interp(expr, funDefs)))
    interp(body, funDefs)

  case Id(name) => sys.error("found unbound id " + name)

  case App(fun, arg) => funDefs(fun) match {
    case FunDef(param, body) =>
      interp(subst(body, param, Num(interp(arg, funDefs))), funDefs)
  }
}
```

Discussion on Namespaces #1

What is the result of interpreting `(f 10)`, where `(f n) = (n n)`?

Also consider `(let f = 1 in (f 2))` ?

Discussion on Namespaces #1

What is the result of interpreting `(f 10)`, where `(f n) = (n n)`?

- Should the interpreter substitute **`n`** in the function position of the application with **`10`**?

```
(f 10)
 -> (interp (subst (app "n" (id "n")) "n" 10))
 -> (interp (app 10 10))
```

  - What would happen if the interpreter does so?
  - Complain that no such function can be found?
  - Complain that the lookup fails because the names of functions must be identifiers, not numbers?

- Or should names for functions and arguments live in separate "spaces" ==> an identifier in the function position of an app-expr is not replaced

Discussion on Namespaces #1

- **Single namespace** (e.g. Scheme). The name of a function can be bound to a value in a local scope, thereby rendering the function inaccessible through that name.

- **Multiple namespaces** (e.g., Common Lisp).
  - The interpreter decides that function names and function arguments live in two separate “spaces”
  - context determines in which space to look up a name.
  - An identifier in a function position within an app-expr is not replaced.

Which Approach Does F1LAE Implement?

```
def interp(expr: F1LAE, funDefs: Map[String, FunDef] ): Int = expr match {
  ...

  case Let(id, expr, body) =>
    val body = subst(body, id, Num(interp(expr, funDefs)))
    interp(body, funDefs)

  case Id(name) => sys.error("found unbound id " + name)

  case App(fun, arg) => funDefs(fun) match {
    case FunDef(param, body) =>
      interp(subst(body, param, Num(interp(arg, funDefs))), funDefs)
  }
}
```

```
interp(
   App("f", 10),
   Map("f" -> FunDef("n", App("n", Id("n"))))
)
```

F1LAE keeps multiple namespaces.

- Identifiers (incl. function arguments) and function names are “looked-up” differently
- Identifiers are not looked-up at all but immediately substituted as soon as a corresponding binding instance is found for them.

Discussion on Namespaces #2

What is the result of `{f 10}` where `{f x} = {g {+ x 3}}`
`{g y} = {- y 1}`

- Should `f` be able to invoke `g`, or should the invocation fail because `g` is defined after `f`?
- What if there are multiple bindings for the same name?
- If a function can invoke every defined function, it can also invoke itself. Do we have recursion in F1LAE?

Discussion on Namespaces #2

What is the result of `{f 10}` where `{f x} = {g {+ x 3}}`
`{g y} = {- y 1}`

- The answer is: 12.
- F1LAE allows each function to see each other function's definition and each name is bound once. It is, however, possible to have other models.
- Question concerning recursion: The F1LAE will enter an endless loop because the language has no means to stop the recursion – no conditionals. But, if it had, there should not be a problem.

Outline

- Implementing first-order functions
- **Environments (static vs. dynamic scoping)**
- About first-class functions
- Functional decomposition and recursion patterns

`let` and Substitution

- Recall from last lecture: Substitution-based evaluation is slow

- To reduce the following expression, the interpreter has to apply substitution three times: once for each let.

```
(let (x 3)
   (let (y 4)
      (let (z 5)
         (x + (y + z)))))
```

- If the program has size **n** (measured in abstract syntax tree nodes), then each substitution sweeps the rest of the program once, making the complexity of this interpreter $O(n^2)$.

- This is rather wasteful. Can we do better?

`let` and Environments

- The interpreter receives a store called **environment,** which maps identifiers to values. We represent environments as values of the type **`Env`**

```
type Env = Map[String, Int]
```

- **`(let (x e) …)`** stores in the environment a mapping from **`x`** to the value of **`e`**
- When the interpreter encounters an **`id`** expression, it looks up the corresponding value in the environment
- Free variables not in environment → error

Here is F1LAE. How Does it Change?

```
def interp(expr: F1LAE, funDefs: FunDefs): Int = expr match {
  case Num(n) => n
  case Add(lhs, rhs) =>
    interp(lhs, funDefs) + interp(rhs, funDefs)
  case Sub(lhs, rhs) =>
    interp(lhs, funDefs) - interp(rhs, funDefs)

  case Let(id, expr, body) =>
    val body = subst(body, id, Num(interp(expr, funDefs)))
    interp(body, funDefs)

 case Id(name) =>
    sys.error("found unbound id " + name)

 case App(fun, arg) => funDefs(fun) match {
   case FunDef(param, body) =>
    interp(subst(body, param, Num(interp(arg, funDefs))), funDefs)
  }
}
```

Is this our F1LAE with Environments?

```
def interp(expr: F1LAE, funDefs: FunDefs, env: Env): Int =
 expr match {
   case Num(n) => n
   case Add(lhs, rhs) =>
         interp(lhs, funDefs, env) + interp(rhs, funDefs, env)
   case Sub(lhs, rhs) => ...

   case Let(id, expr, body) =>
    val newEnv = env + (id -> interp(expr, funDefs, env))
    interp(body, funDefs, newEnv)

   case Id(name) =>  env(name)

   case App(fun, arg) => funDefs(fun) match {
    case FunDef(param, body) =>
     val funEnv = env + (param -> interp(arg, funDefs, env))
              interp(body, funDefs, funEnv)
   }
}
```

What's the result of evaluating:

(let (n 5) (f 10)) where f(x) = n

Is the result the same when using F1LAE with substitution?

Is this our F1LAE with Environments?

```
def interp(expr: F1LAE, funDefs: FunDefs, env: Env): Int =
 expr match {
   ...

   case App(fun, arg) => funDefs(fun) match {
    case FunDef(param, body) =>
     val funEnv = env + (param -> interp(arg, funDefs, env))
              interp(body, funDefs, funEnv)
   }
}
```

The result is 5, which is not the same as what we would get with substitution. The latter is our reference!

The result of applying a function to a certain parameter may change depending on the calling context?

This contradicts our mathematical understanding of a function.

Static Versus Dynamic Scoping

Definition Scope (of a name binding): The scope of a name binding is the part of the program where the binding is in effect.

Definition Static/Lexical Scoping: The scope of a name binding is determined syntactically (at compile-time).

Definition Dynamic Scoping: The scope of a name binding is determined by the execution context (at runtime).

Reconsider F1LAE… What strategy does it implement?

It implements dynamic scoping. How to get static scoping?

F1LAE with Environments

```
def interp(expr:F1LAE, funDefs:FunDefs, env:Env): Int = expr match {
  case Num(n) =>
    n
  case Add(lhs, rhs) =>
    interp(lhs, funDefs, env) + interp(rhs, funDefs, env)
  case Sub(lhs, rhs) =>
    interp(lhs, funDefs, env) - interp(rhs, funDefs, env)

  case Let(id, expr, body) =>
    val newEnv = env + (id -> interp(expr, funDefs, env))
    interp(body, funDefs, newEnv)

  case Id(name) =>
    env(name)

  case App(fun, arg) => funDefs(fun) match {
    case FunDef(param, body) =>
      val funEnv = ~~env~~ + Map(param -> interp(arg, funDefs, env))
      interp(body, funDefs, funEnv)
  }
}
```

Static Scoping!

Outline

- Implementing first-order functions
- Environments (static vs. dynamic scoping)
- **About first-class functions**
- Functional decomposition and recursion patterns

Abstracting over Computations

- What does it mean to abstract over computations?
- Why is it meaningful? Give examples.

Consider the process of filtering the elements of a list according to some criteria - any function on the elements of the list…

It makes sense to avoid writing a specific filter function for any potential criteria …

Abstracting With Functions as Parameters

What does this code do?

```
def filter1[A, B] (relOp: (A, B) => Boolean, b: B, list: List[A]): List[A] =
    list match {
      case Nil => Nil
      case x :: xs =>
        val filteredRest = filter1(relOp, b, xs)
        if (relOp(x, b)) x :: filteredRest
        else filteredRest
  }
```

`relOp` parameter stands for any relational operation to apply

```
def <(a: Int, b: Int) = a < b
def >(a: Int, b: Int) = a > b
def squaredGt(x: Int, c: Int) = x * x > c

def below(thres: Int, l: List[Int]) = filter1(<, thres, l)
def above(thres: Int, l: List[Int]) = filter1(>, thres, l)

println(below(4, List(1, 2, 3, 4, 5)))
println(above(4, List(1, 2, 3, 4, 5)))
println(filter1(squaredGt, 10, List(1, 2, 3, 4, 5)))
```

Functions that Return Functions

- **Expressions in functional languages (Scheme, Haskell, Scala) can evaluate to functions.**
- The body of a function is an expression
- Hence, **a function can return a function**.

Especially useful when the produced function "remembers" arguments …

```
def add(x: Int) = {
  def xAdder(y: Int) = x + y
  xAdder _
}
```

The result of **`add`** "remembers" the value of the argument **`x`**.
Every time we use the result of applying **`add`**, it uses the value of **`x`** at application time of **`add`**

Functions that Return Functions

```
def filter2[A, B](relOp: (A, B) => Boolean) = {
  def innerFilter(b: B, list: List[A]): List[A] = list match {
    case Nil => Nil
    case x :: xs => {
      val filteredRest = innerFilter(b, xs)
      if (relOp(x, b)) x :: filteredRest
      else filteredRest
    }
  }

  innerFilter _
}
```

`filter2` consumes **`relOp`**, defines **`innerFilter`**, and returns it. **`innerFilter`** remembers **`relOp`**

```
def filterBelow = filter2(<)
def filterAbove = filter2(>)
def filterSquare = filter2(squaredGt)
```

Anonymous Functions

- If auxiliary functions are only used as arguments to some abstract function **`f`**, we use an inner **`def`**.

```
case class Person(name: String)

def findAnon(list: List[Person], name: String) = {
  def hasName(p: Person, name: String) = p.name == name
  filter(hasName, name, list)
}
```

- Anonymous functions provide a short-hand for this frequent use of locally defined functions → **`lambda`**-expressions

- **`lambda`**-expressions are used to create functions in the same way as **`def`**, except that no function name is specified

Syntax of lambdas in Scala

```
(<var>, … <var>) => exp
```

- The sequence of variables in parentheses are the function's parameters
- The **`exp`** on the right-hand side is the function's body

```
(x_1, …, x_n) => exp
```

⇕

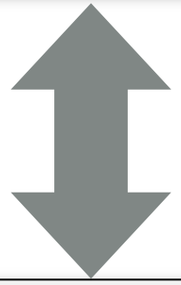
```
def aNewName(x_1, …, x_n) = exp
```

`aNewName` may not occur in **`exp`**

```
def plus4(x: Int) = x + 4
def plus4 = (x: Int) => x + 4
```

Syntax of lambdas in Scala

```scala
case class Person(name: String)

def findAnon(list: List[Person], name: String) = {
  def hasName(p: Person, name: String) = p.name == name
  filter(hasName, name, list)
}
```



```scala
def findAnon(list: List[Person], name: String) = {
 filter(
   (p: Person, name: String) => { p.name == name },
   name,
   list)
}
```

Anonymous Functions

```
def filter2[A, B](relOp: (A, B) => Boolean) = {
  def innerFilter(b: B, list: List[A]): List[A] = list match {
    case Nil => Nil
    case x :: xs => {
      val filteredRest = innerFilter(b, xs)
      if (relOp(x, b)) x :: filteredRest
      else filteredRest
    }
  }
  innerFilter _
}
```

Can we use an anonymous function here?

Outline

- Implementing first-order functions
- Environments (static vs. dynamic scoping)
- About first-class functions
- **Functional decomposition and recursion patterns**

How Do You (Dis)like `list-of-squares`?

```
def listOfSquares(list:List[Int]): List[Int] = list match {
   case Nil => Nil
   case x :: xs => square(x) :: listOfSquares(xs)
}
```

- The definition draws attention to the element-by-element processing of the input list.
- The operation applied to elements (`square`) is hard-coded.
- How the result is constructed from the input is hard-coded (coupled to a particular implementation of the list).

What About This Definition?

```
def listOfSquares2(list: List[Int]) = map(square, list)

def map[A, B](f:A => B, list:List[A]): List[B] = list match {
   case Nil => Nil
   case x :: xs => f(x) :: map(f, xs)
}
```

This version emphasizes squaring as a transformation of a list to another list

Recursion and Induction

- Many programs that involve repetition are naturally expressed using recursion
- Proofs of properties of such programs are naturally expressed using some form of induction.

- In functional programming, recursion and induction are primary tools for defining and proving properties of programs.

Recursion Operators

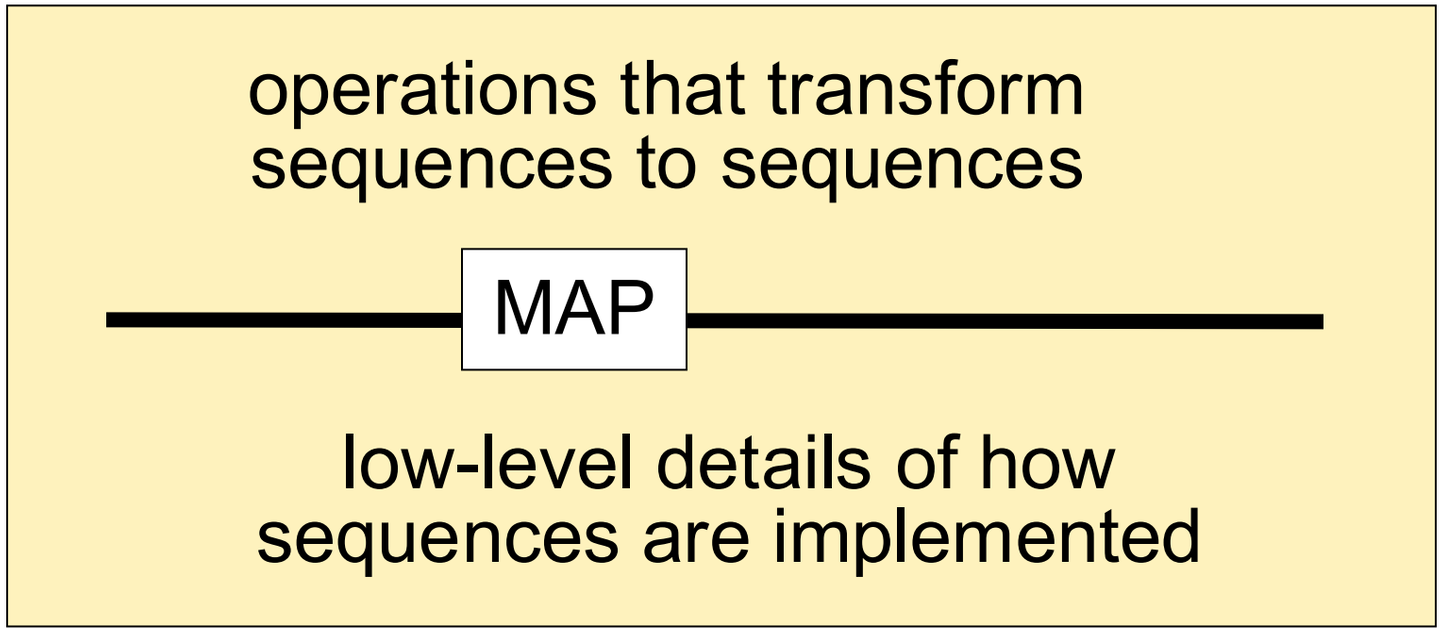
- Unsurprisingly, many recursive programs share a **common pattern of recursion**.
- Repeating the same patterns again and again is tedious, time consuming, error prone.

- Such repetition can be avoided by introducing special recursion operators.
- **Recursion operators** encapsulate common patterns allowing to concentrate on parts that are different for each application.

`map`: an Abstraction Barrier

- `map` encapsulates a recursion pattern.

- **Abstraction barrier**: `map` supports a higher-level of abstraction
- It isolates the implementation of procedures that transform lists from the details of how list elements are extracted and combined.
- One can vary the implementation of the list independent of the mapping function applied to each element.



Quiz

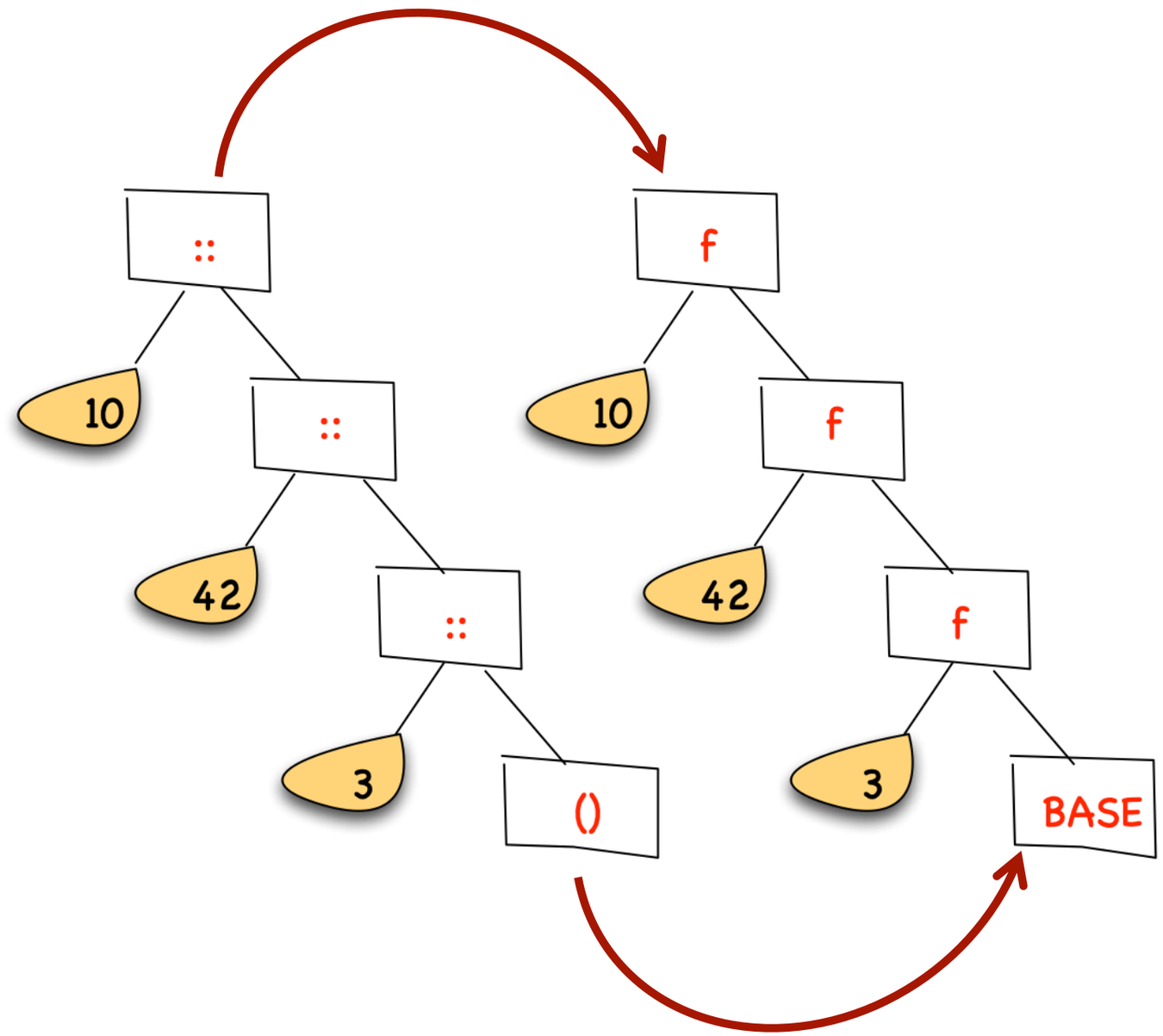
Can you think of map as an instance of another recursion operator?

```
def map[A, B](f: A => B, list: List[A]): List[B] =
 list match
 {
   case Nil => Nil
   case x :: xs => f(x) :: map(f, xs)
 }
```

The `fold`*) Recursion Operator

*) aka **reduce**, **accumulate**, **compress** or **inject**

`fold` encapsulates a pattern of recursion for processing lists, in which the two constructors for lists are replaced by other values and functions.



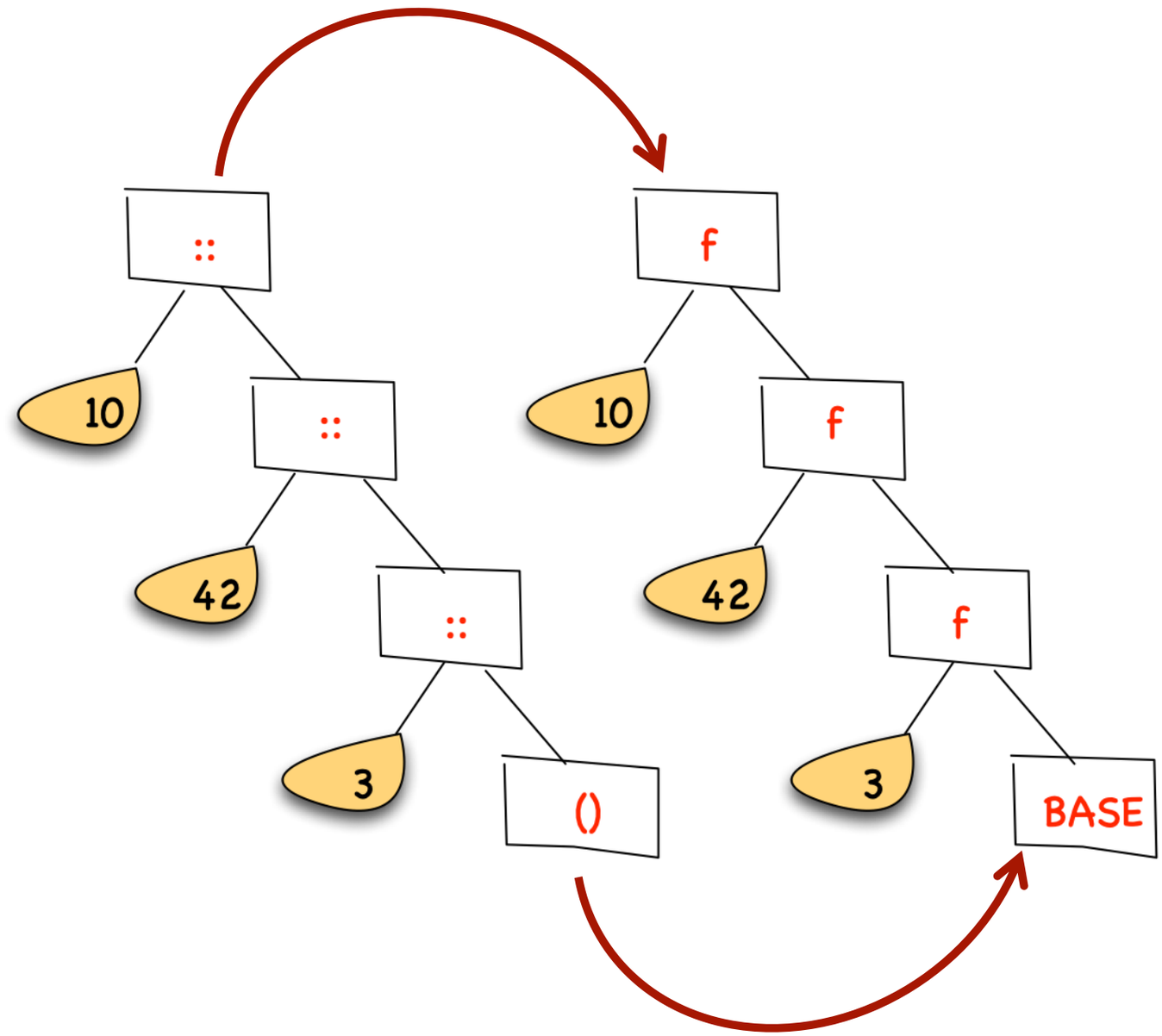
Go watch:
https://vimeo.com/64673035

map as fold?

```
def map[A, B](f: A=>B, list: List[A]): List[B] =
 list match
 {
   case Nil => Nil
   case x :: xs => f(x) :: map(f, xs)
 }
```

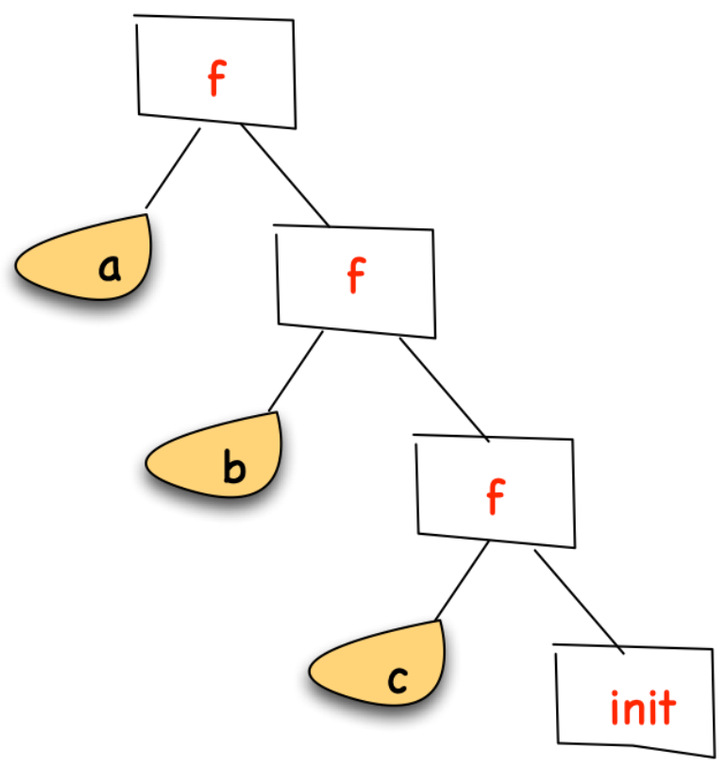
What is BASE?
What is f?

base = ()
f = (x ys) => (f x) :: ys)



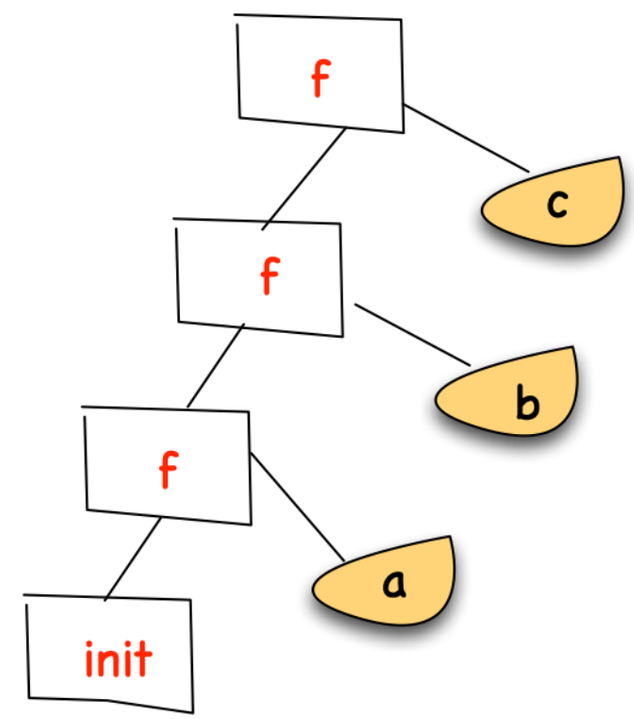
`fold` in Scala

```
def fold[A, B](init: B, combine: (A, B) => B, l: List[A]): B
= l match {
   case Nil => init
   case x :: xs => combine(x, fold(init, combine, xs))
 }
```



Two predefined operations:

`foldRight, foldLeft`




```
List(a, b, c).foldRight(init)(f)
==
f(a, f(b, f(c, init)))
```

```
List(a, b, c).foldLeft(init)(f)
==
f(f(f(init a), b), c)
```

Some Instantiations of `fold`

```
def myMap[A, B] (f: A => B, list: List[A]): List[B] = {
  list.foldRight(List())((x: A, l: List[B]) => (f(x) :: l))
}

println(myMap((x: Int) => 2*x, List(1, 4, 5) ))
println(myMap((x: Int) => (x + 1), List(1, 2, 3)))
```

Other Instantiations of `fold`

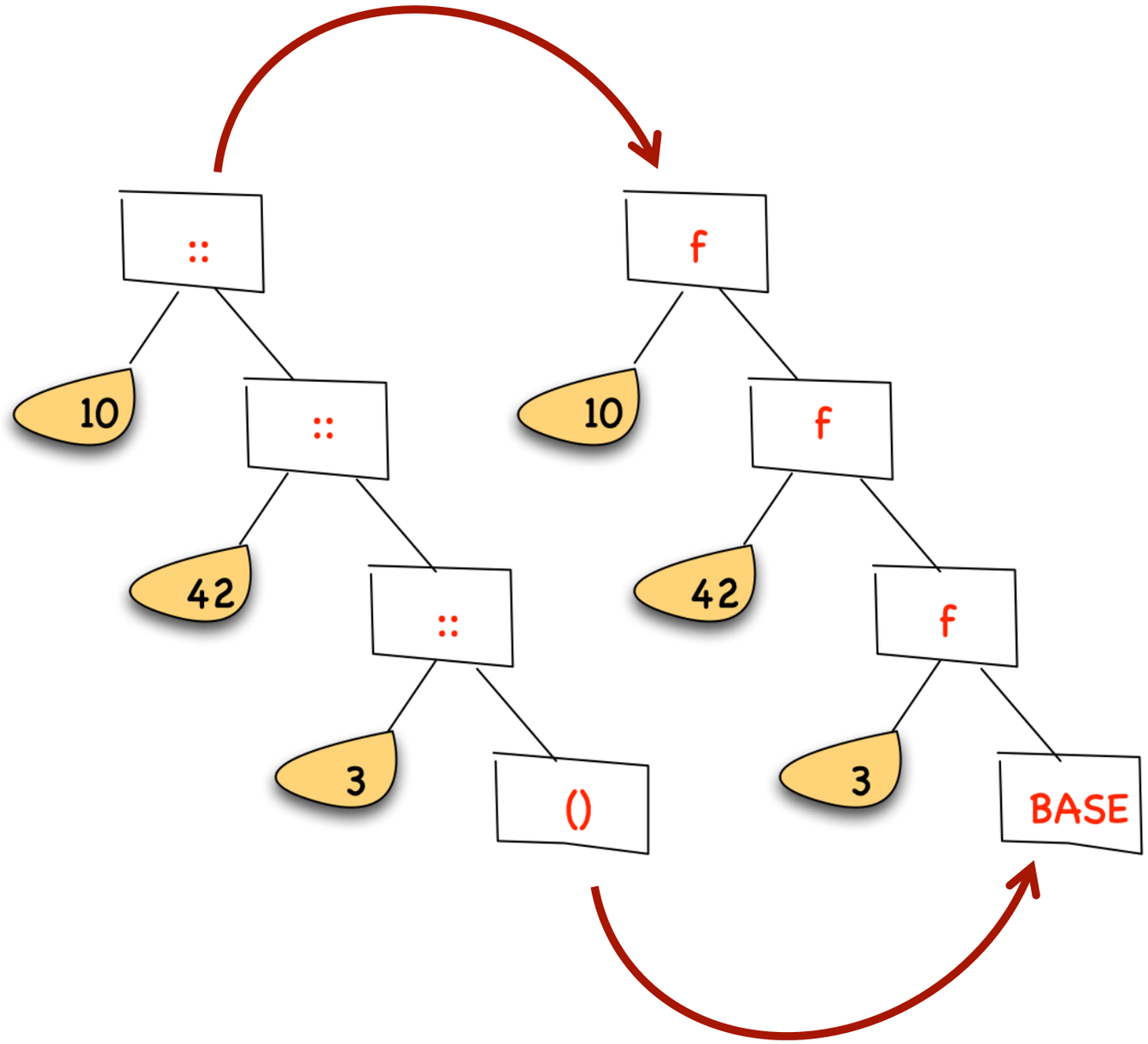
```
package templates

object FoldingTemp extends App {
  def summing(list: List[Int]): Int = ???
  def product(list: List[Int]): Int = ???
  def length[A](list: List[A]): Int = ???
  def reverse[A](list: List[A]): List[A] = ???
  def myMap[A, B] (f: A => B, list: List[A]): List[B] = ???
  def myFilter[A](p: A => Boolean, list: List[A]): List[A] = ???

  println(summing(List(1, 4, 5)))
  println(product(List(1, 4, 5)))
  println(length(List(1, 4, 5)))
  println(reverse(List(1, 4, 5)))
  println(myMap( (x:Int) => 2*x, List(1, 4, 5) ))
  println(myFilter( (x: Int) => x > 2, List(1, 4, 5) ) )
}
```

The `fold`*) Recursion Operator

*) aka **reduce**, **accumulate**, **compress** or **inject**

Origins in recursion theory (Kleene, 1952).

Its use as a central concept in programming languages dates back to the reduction operator of APL (Iverson, 1962), and the insertion operator of FP (Backus, 1978).



It may take a long time until good ideas catch up in practice, but eventually this happens

When Process Structure Follows Data …

```
case class Tree(value: Int, branches: List[Tree])

def sumOfOddSquares(forest: List[Tree]): Int = forest match {
  case Nil => 0
  case x :: xs => x match {
    case Tree(value, branches) =>
      val restSum = sumOfOddSquares(branches) + sumOfOddSquares(xs)
      if (isOdd(value)) square(value) + restSum
      else restSum
  }
}
```

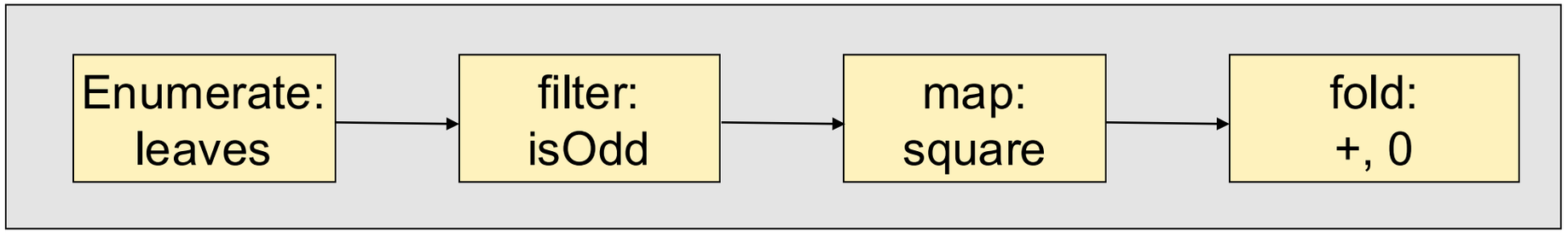
```
def evenFibs(n: Int): List[Int] = {
  def process(k: Int): List[Int] = {
    if (k > n) Nil
    else {
      val f = fib(k)
      if (isEven(f)) f :: process(k + 1)
      else process(k + 1)
    }
  }
  process(0)
}
```

Do you recognize any similarity between these two processes?

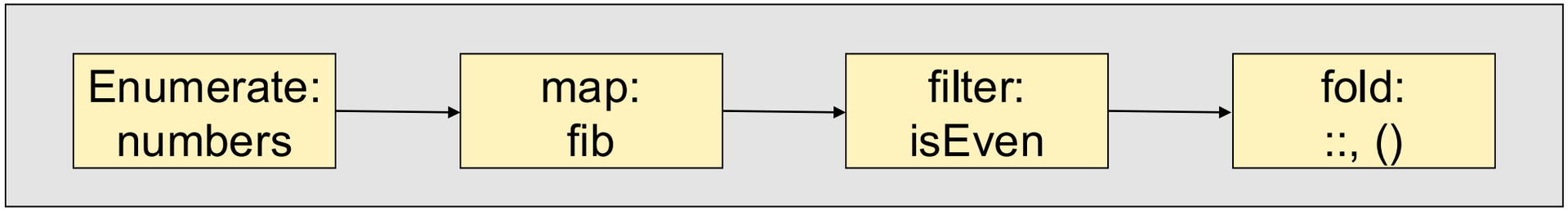
Focusing on the Process Structure…

- **`sumOfOddSquares`**:
  - enumerates the leaves of a tree
  - filters them to select the odd ones **generic filter**
  - process each of the selected ones by squaring **generic map**
  - folds the result list using +, starting with 0 **generic fold**
- **`evenFibs`**:
  - enumerates the integers from 0 to n
  - process each integer by computing its fibonacci **generic map**
  - filters them to select the even ones **generic filter**
  - folds the result list using ::, starting with empty list **generic fold**

sum-of-odd-squares



list-of-even-fibs



Advantages of Functional Decomposition

Functional decomposition supports **program designs that are modular** by combining independent components, instantiated from recursion operators that can be provided in a library of standard components.

Modular construction is a powerful strategy for **controlling complexity** in engineering design